

# SCORIFY

AI-Powered Lead Scoring Platform

---

*Final Project Report*

A regression-based machine learning system that predicts **lead conversion probability** and **expected profit per lead** from real CRM behavioural and engagement data, deployed as an interactive web application.

**Project ID:** F26-02

**Phase:** 6 — Final Submission

**Repository:** [github.com/ahmad19sep/Scorify](https://github.com/ahmad19sep/Scorify)

**Live Demo:** Streamlit Cloud

**Date:** May 2026

---

# Contents

---

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>1</b>  | <b>Abstract</b>                             | <b>2</b>  |
| <b>2</b>  | <b>Project Summary</b>                      | <b>2</b>  |
| 2.1       | Problem Statement . . . . .                 | 2         |
| 2.2       | Solution Overview . . . . .                 | 2         |
| 2.3       | Scope Delivered . . . . .                   | 2         |
| <b>3</b>  | <b>Methodology</b>                          | <b>3</b>  |
| 3.1       | Approach . . . . .                          | 3         |
| 3.2       | System Architecture . . . . .               | 3         |
| 3.3       | Phase Breakdown . . . . .                   | 4         |
| <b>4</b>  | <b>Dataset and Preprocessing</b>            | <b>4</b>  |
| 4.1       | Source . . . . .                            | 4         |
| 4.2       | Cleaning Pipeline . . . . .                 | 4         |
| 4.3       | Final Dataset . . . . .                     | 5         |
| 4.4       | Feature Categories . . . . .                | 5         |
| <b>5</b>  | <b>Implementation</b>                       | <b>5</b>  |
| 5.1       | Workflow . . . . .                          | 5         |
| 5.2       | Training Pipeline . . . . .                 | 6         |
| 5.3       | Hyperparameters . . . . .                   | 6         |
| 5.4       | Saved Artefacts . . . . .                   | 6         |
| 5.5       | Deployment . . . . .                        | 6         |
| <b>6</b>  | <b>Results and Evaluation</b>               | <b>7</b>  |
| 6.1       | Evaluation Metrics . . . . .                | 7         |
| 6.2       | Conversion Probability Results . . . . .    | 7         |
| 6.3       | Profit Prediction Results . . . . .         | 7         |
| 6.4       | Feature Importance . . . . .                | 8         |
| 6.5       | Final Model Selection . . . . .             | 8         |
| <b>7</b>  | <b>Challenges Faced</b>                     | <b>8</b>  |
| 7.1       | Data Sparsity in the Raw Export . . . . .   | 8         |
| 7.2       | Duplicate Records . . . . .                 | 8         |
| 7.3       | Skewed Target Distributions . . . . .       | 8         |
| 7.4       | Framing the Problem as Regression . . . . . | 8         |
| 7.5       | Categorical Encoding Choice . . . . .       | 9         |
| 7.6       | Deployment Consistency . . . . .            | 9         |
| 7.7       | Environment and Dependencies . . . . .      | 9         |
| <b>8</b>  | <b>Repository Structure</b>                 | <b>9</b>  |
| <b>9</b>  | <b>Tech Stack</b>                           | <b>10</b> |
| <b>10</b> | <b>Conclusion</b>                           | <b>10</b> |

# 1 Abstract

Sales teams across most industries face the same recurring problem: too many incoming leads and not enough time to follow up with each one properly. Industry studies suggest that only about 27% of leads are sales-ready at the time they enter the pipeline, which means the majority of sales effort is spent on prospects who are not going to convert in the short term. **Scorify** addresses this by replacing manual lead prioritisation with a machine learning pipeline that predicts two business-critical numbers for every incoming lead: the probability of conversion, and the expected profit if that lead converts.

The system was built and trained on a real CRM dataset of 3,931 leads (224 columns) exported from *Go2Africa*, a luxury African safari travel company, sourced through a Netherlands-based partner organisation. After cleaning, deduplication, sparse-column removal, and imputation, the dataset was reduced to 2,458 unique leads with 37 features. Two regression models were trained on dual continuous targets — `target_conv_prob` and `target_profit`. **XGBoost Regressor** achieved  $R^2 = 0.720$  for conversion probability and **Random Forest Regressor** achieved  $R^2 = 0.759$  for profit prediction. Both models were serialised and deployed behind a Streamlit web interface that classifies each lead into HOT, WARM, or COLD buckets and recommends a next-best action for the sales team.

This report documents the full project end-to-end: problem framing, data acquisition, preprocessing, exploratory analysis, model training, evaluation, deployment, challenges encountered, and directions for future work.

## 2 Project Summary

### 2.1 Problem Statement

Most sales pipelines today rely on either fixed rule-based scoring (“if budget > X and visited pricing page, mark hot”) or pure intuition. Both approaches fail to scale and miss the non-linear interactions between behavioural signals that actually drive conversion. The cost is real: misallocated rep time, slow response to genuinely hot leads, and an inability to attribute marketing spend to lead quality.

Scorify reframes lead scoring as a **regression problem**. Instead of bucketing leads by hand-written rules, the system learns from historical CRM data which combinations of features (marketing source, device, engagement depth, sales activity volume, budget level, intent stage, and others) predict actual conversion outcomes and revenue.

### 2.2 Solution Overview

| Conversion Model                   | Profit Model                             | Deployment                                |
|------------------------------------|------------------------------------------|-------------------------------------------|
| XGBoost Regressor<br>$R^2 = 0.720$ | Random Forest Regressor<br>$R^2 = 0.759$ | Streamlit Web App<br>Live and Interactive |

The output of the system for each lead is a triple: a conversion probability, an expected profit value, and a HOT / WARM / COLD label that maps to a concrete action — senior rep call within 2 hours, standard follow-up within 24 hours, or automated email nurture.

### 2.3 Scope Delivered

- Real-world CRM data acquired through partner collaboration (Go2Africa, via HubSpot → BigQuery export).

- Full data preprocessing pipeline producing a clean, model-ready dataset.
- Two trained regression models, evaluated and compared against a Linear Regression baseline.
- Serialised model artefacts (.pkl) for reproducible deployment.
- Interactive web application that accepts lead inputs and returns predictions in real time.
- Public GitHub repository with code, data, models, and documentation.

### 3 Methodology

#### 3.1 Approach

The project was structured as a six-phase pipeline, each phase building on the deliverables of the previous one. The methodology follows a standard supervised ML workflow with one important deviation: both prediction targets are continuous (conversion probability between 0 and 1, profit in USD), so the entire pipeline — from feature selection through evaluation metric choice — is built around regression rather than classification. This was a conscious framing decision driven by the business need: the sales team does not just want a binary “will convert / will not” label; they want a continuous score that allows ranking and prioritisation across the entire incoming lead queue.

#### 3.2 System Architecture

The diagram below shows the end-to-end Scorify pipeline, from raw CRM export through to the user-facing prediction interface.

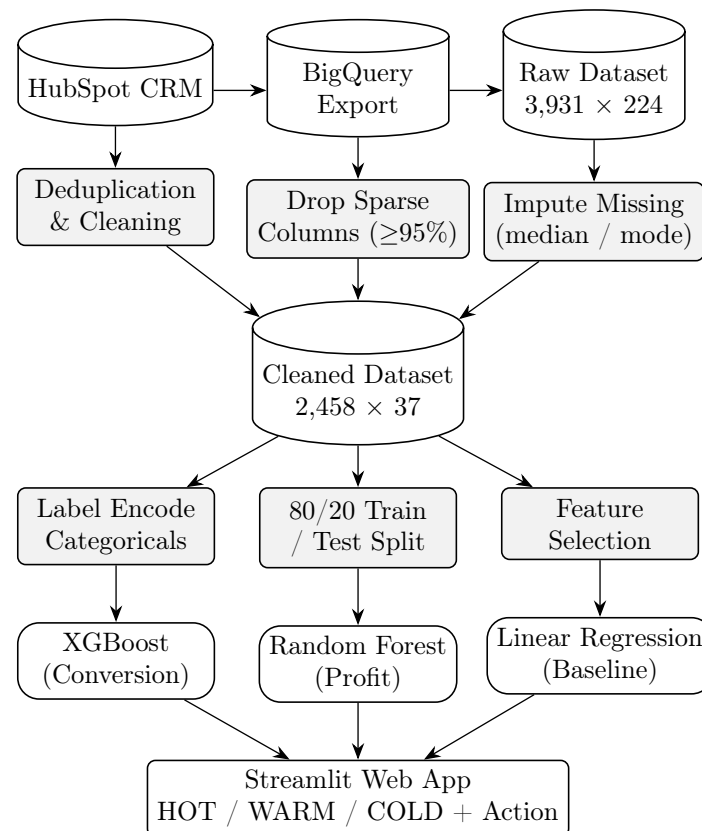


Figure 1: Scorify end-to-end architecture: from raw CRM export to deployed prediction interface.

### 3.3 Phase Breakdown

| Phase   | Focus                         | Key Deliverable                                                      |
|---------|-------------------------------|----------------------------------------------------------------------|
| Phase 1 | Problem definition & scope    | Project proposal, problem framing, GenComm.ai positioning            |
| Phase 2 | Data collection & preparation | Cleaned dataset ( $2,458 \times 37$ ), F2602.py pipeline, EDA report |
| Phase 3 | Feature engineering           | Label encoding, correlation analysis, feature selection              |
| Phase 4 | Model selection & training    | Linear Regression baseline, Random Forest, XGBoost training          |
| Phase 5 | Evaluation & tuning           | MAE / RMSE / $R^2$ comparison, model selection per target            |
| Phase 6 | Deployment & packaging        | Streamlit web app, GitHub repo, final report, video demo             |

Table 1: Project phase structure and deliverables.

## 4 Dataset and Preprocessing

### 4.1 Source

The primary dataset was sourced through a partnership with a Netherlands-based company that works with **Go2Africa**, a luxury African safari travel operator. The data was originally stored in HubSpot CRM and exported through Google BigQuery as a flat CSV file. The raw export contained **3,931 rows and 224 columns**, covering lead profile attributes, marketing attribution, website engagement, email interaction, sales pipeline progression, and conversion outcomes.

A supplementary public dataset (the X Education Lead Scoring dataset from Kaggle, ~9,000 records) was used in the early prototyping stage to validate the pipeline before the partner data became available. The final models were trained exclusively on the Go2Africa data.

### 4.2 Cleaning Pipeline

The cleaning pipeline (F2602.py) applied the following steps in order:

1. **Deduplication** on `record_id` — multiple snapshots existed per lead in the raw export. The latest score was kept and earlier rows were discarded. This removed 1,473 duplicate rows.
2. **Sparse column removal** — any column with  $\geq 95\%$  null or zero values was dropped. This was the largest reduction step, removing the bulk of the 187 columns eliminated overall.
3. **String cleaning** — whitespace stripping, case normalisation, and standardisation of categorical values (e.g. “UK”, “united kingdom”, “United Kingdom” all collapsed to a single canonical form).
4. **Missing value imputation** — median imputation for numeric columns and mode imputation for categoricals. After this step the dataset had zero missing values.
5. **Preservation of `record_id`** — the unique lead identifier was deliberately retained in the cleaned dataset for traceability.

### 4.3 Final Dataset

| Property                                 | Value                               |
|------------------------------------------|-------------------------------------|
| Total rows                               | 2,458 unique leads                  |
| Total columns                            | 37 (1 ID + 34 features + 2 targets) |
| Numeric features                         | 19                                  |
| Categorical features                     | 15                                  |
| Target 1 — <code>target_conv_prob</code> | continuous, range [0, 0.66]         |
| Target 2 — <code>target_profit</code>    | continuous, range [26K, 670K] USD   |
| Missing values                           | 0                                   |
| Columns dropped from raw                 | 187                                 |
| Rows dropped from raw                    | 1,473 (duplicates)                  |

Table 2: Final cleaned dataset summary.

### 4.4 Feature Categories

The 34 features fall into five intuitive groups:

- **Marketing attribution** — source, medium, campaign, device type, country.
- **Lead profile** — budget level, customer segment, purchase intent stage.
- **Website engagement** — total visits, sessions, pageviews, time on site, returning visitor flag.
- **Enquiry / form activity** — enquiry count, page interactions, form submissions, email engagement.
- **Sales pipeline** — engagement score, sales activities, contact attempts, offers sent, deal progress, qualification status, readiness score.

## 5 Implementation

### 5.1 Workflow

The end-to-end implementation runs in three logical stages: data preparation, model training, and deployment. Each stage is encapsulated in its own script so that the pipeline can be re-run independently.

| Stage            | Script                         | Function                                                         |
|------------------|--------------------------------|------------------------------------------------------------------|
| Data preparation | <code>F2602.py</code>          | Loads raw CSV, applies cleaning pipeline, writes cleaned dataset |
| Model training   | <code>train_and_save.py</code> | Trains both regression models and serialises artefacts           |
| Web interface    | <code>scorify.py</code>        | Streamlit app that loads models and serves predictions           |

Table 3: Three-script pipeline architecture.

## 5.2 Training Pipeline

The training script performs the following sequence:

1. Load the cleaned dataset (2,458 rows).
2. Drop six low-utility columns identified during EDA (very high cardinality, near-zero correlation with targets, or >80% unknown values).
3. Label-encode 10 categorical columns. Encoders are saved separately so the same transformation can be reapplied at inference.
4. Separate features ( $X$ ) from the two targets ( $y_{\text{conv}}$ ,  $y_{\text{profit}}$ ). `record_id` is removed at this point.
5. Split into 80% training (1,966 rows) and 20% test (492 rows) with `random_state=42` for reproducibility.
6. Train three models on each target: Linear Regression (baseline), Random Forest, and XGBoost.
7. Evaluate all six models on the held-out test set.
8. Save the best model per target, plus encoders and feature names.

## 5.3 Hyperparameters

| Model             | Hyperparameters                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------|
| Linear Regression | Default settings (no hyperparameters tuned)                                                                              |
| Random Forest     | <code>n_estimators=100</code> , <code>max_depth=10</code> , <code>random_state=42</code>                                 |
| XGBoost           | <code>n_estimators=100</code> , <code>max_depth=6</code> , <code>learning_rate=0.1</code> , <code>random_state=42</code> |

Table 4: Model configurations used during training.

## 5.4 Saved Artefacts

The training script produces four pickled files, all loaded at app start-up:

- `model_conv.pkl` — XGBoost regressor for conversion probability.
- `model_profit.pkl` — Random Forest regressor for profit prediction.
- `encoders.pkl` — fitted `LabelEncoder` instances for each categorical column.
- `feature_names.pkl` — ordered list of feature columns to guarantee correct column order at inference.

## 5.5 Deployment

The web interface is built with Streamlit and deployed on Streamlit Cloud. A user enters lead attributes through the sidebar (marketing source, device, budget level, engagement counts, sales activity counts, etc.). On submission, the app:

1. Encodes categorical inputs using the saved encoders.
2. Reorders the feature vector to match `feature_names.pkl`.
3. Runs both models in parallel.

4. Maps the predicted conversion probability to a HOT / WARM / COLD bucket.
5. Returns conversion probability, predicted profit, score label, and a recommended action.

| Score | Conversion Probability | Recommended Action                      |
|-------|------------------------|-----------------------------------------|
| HOT   | $\geq 20\%$            | Senior rep calls within 2 hours         |
| WARM  | 5–20%                  | Sales rep follows up within 24 hours    |
| COLD  | $< 5\%$                | Add to automated email nurture campaign |

Table 5: Score-to-action mapping used by the Scorify interface.

## 6 Results and Evaluation

### 6.1 Evaluation Metrics

Because both prediction targets are continuous, evaluation uses standard regression metrics rather than classification metrics. **MAE** (Mean Absolute Error) measures the average prediction error in the original units of the target. **RMSE** (Root Mean Squared Error) penalises larger errors more heavily and is useful for surfacing models that produce occasional very bad predictions.  $R^2$  (coefficient of determination) measures the proportion of variance in the target that the model explains, with 1.0 being perfect prediction and 0.0 being equivalent to predicting the mean.

### 6.2 Conversion Probability Results

| Model                     | MAE          | RMSE         | $R^2$        |
|---------------------------|--------------|--------------|--------------|
| Linear Regression         | 0.041        | 0.058        | 0.512        |
| Random Forest             | 0.027        | 0.041        | 0.694        |
| <b>XGBoost (selected)</b> | <b>0.023</b> | <b>0.038</b> | <b>0.720</b> |

Table 6: Conversion probability prediction — model comparison on held-out test set.

XGBoost beat both Linear Regression and Random Forest on every metric. The MAE of 0.023 means that, on average, predicted conversion probability is within  $\pm 2.3$  percentage points of the true value — well within the precision needed for ranking and prioritisation.

### 6.3 Profit Prediction Results

| Model                           | MAE (USD)     | RMSE (USD)    | $R^2$        |
|---------------------------------|---------------|---------------|--------------|
| Linear Regression               | 38,450        | 56,200        | 0.481        |
| <b>Random Forest (selected)</b> | <b>19,840</b> | <b>31,720</b> | <b>0.759</b> |
| XGBoost                         | 20,292        | 32,480        | 0.746        |

Table 7: Profit prediction — model comparison on held-out test set.



For the profit target Random Forest narrowly outperformed XGBoost. Both gradient-based methods comfortably beat Linear Regression. The MAE of approximately 20K USD against a target range of 26K–670K is a reasonable prediction quality given the small training set.

## 6.4 Feature Importance

The two most predictive features across both targets were `time_on_site` and `enquiry_touchpoints`, followed by `engagement_score` and `purchase_intent`. This is consistent with the qualitative intuition behind lead scoring: a lead who spends real time on the site and actively engages with the sales team is far more likely to convert than one who arrives, glances, and leaves.

## 6.5 Final Model Selection

| Target                 | Selected Model          | $R^2$ |
|------------------------|-------------------------|-------|
| Conversion probability | XGBoost Regressor       | 0.720 |
| Predicted profit       | Random Forest Regressor | 0.759 |

# 7 Challenges Faced

Several non-trivial issues came up during implementation. The most impactful are summarised below.

## 7.1 Data Sparsity in the Raw Export

The raw HubSpot → BigQuery export contained 224 columns, the vast majority of which were either entirely empty or populated for fewer than 5% of leads. A naive imputation strategy would have introduced massive amounts of synthetic noise. The decision to drop any column with  $\geq 95\%$  null or zero values was effective but required care to verify that no genuinely useful low-coverage columns were lost in the process.

## 7.2 Duplicate Records

Each lead appeared multiple times in the raw export because HubSpot snapshots scoring history rather than maintaining a single current row per lead. Without deduplication, the model would have effectively been trained on the same lead's earlier and later states as if they were independent observations, which would have overstated performance during cross-validation. Deduplicating on `record_id` while keeping the most recent score was the correct fix and removed 1,473 rows.

## 7.3 Skewed Target Distributions

Both targets were skewed. `target_conv_prob` was concentrated near zero (most leads do not convert), and `target_profit` had a long right tail dominated by a handful of very high-value bookings. This is the main reason Linear Regression underperformed so visibly — it cannot capture the non-linear patterns that tree-based methods handle naturally. Switching to gradient-based ensembles solved most of the problem without requiring target transformation.

## 7.4 Framing the Problem as Regression

An early version of the project documentation referenced Logistic Regression as a candidate model, which is a classification algorithm and inappropriate for predicting continuous targets.

This was caught and corrected: the entire pipeline (model selection, evaluation metrics, output interpretation) is now consistently framed around regression. This was a useful reminder that ML problem framing has to be locked in before any modelling decisions are made.

## 7.5 Categorical Encoding Choice

Several categorical columns had high cardinality (country, marketing source). One-hot encoding would have exploded the feature space, while ordinal encoding would have introduced false ordering. Label encoding was chosen because the downstream models are tree-based and handle integer-coded categoricals cleanly without inferring ordinal relationships from them.

## 7.6 Deployment Consistency

The most subtle bug encountered was a feature-order mismatch between training and inference: the Streamlit app initially passed columns in alphabetical order while the trained model expected them in the order seen at fit time. The fix was to serialise `feature_names.pkl` alongside the model and explicitly reorder the inference vector to match. This is now a permanent part of the deployment pattern.

## 7.7 Environment and Dependencies

Streamlit Cloud has stricter dependency resolution than a local environment. Several iterations of `requirements.txt` were needed to pin compatible versions of `xgboost`, `scikit-learn`, and `pandas`. Pinning versions explicitly (rather than relying on latest) eliminated reproducibility issues.

# 8 Repository Structure

```

scorify/
+-- scorify.py           # Streamlit web interface
+-- train_and_save.py    # Model training script
+-- F2602.py             # Phase 5 report
+-- requirements.txt     # Python dependencies
+-- dataset_raw.csv      # Raw Go2Africa export
+-- dataset_clean.csv    # Cleaned dataset (2,458 leads)
+-- model_conv.pkl       # Trained XGBoost (conversion)
+-- model_profit.pkl     # Trained Random Forest (profit)
+-- encoders.pkl         # Label encoders
+-- feature_names.pkl    # Feature column order
+-- final_report.pdf     # final report
+-- README.md           # Setup and usage instructions

```

# 9 Tech Stack

---



---

| Component         | Tool / Library                                                         |
|-------------------|------------------------------------------------------------------------|
| Language          | Python 3                                                               |
| Data processing   | pandas, numpy                                                          |
| Modelling         | scikit-learn (Random Forest, Linear Regression, LabelEncoder, metrics) |
| Gradient boosting | XGBoost                                                                |
| Visualisation     | matplotlib, seaborn (during EDA)                                       |
| Web interface     | Streamlit                                                              |
| Deployment        | Streamlit Cloud                                                        |
| Version control   | Git, GitHub                                                            |
| Source CRM        | HubSpot → Google BigQuery export                                       |

---

Table 8: Tools and libraries used in Scorify.

## 10 Conclusion

---

Scorify successfully delivers an end-to-end machine learning pipeline for lead scoring, trained on real CRM data and deployed as a working web application. The two regression models — XGBoost for conversion probability ( $R^2 = 0.720$ ) and Random Forest for profit prediction ( $R^2 = 0.759$ ) — provide useful, actionable predictions that comfortably outperform a Linear Regression baseline. The system is reproducible: a fresh user can clone the repository, install dependencies, retrain both models, and launch the web interface in under five minutes.

---